AI Agents for Optimization of Thin-Shell Composites

ME 295 Lab Report 2

Tara Venkatadri and Federico Benazzo
California Institute of Technology, Pasadena, CA • October 29, 2026

Team: Tara Venkatadri and Federico Benazzo (identical reports are submitted, as permitted in the feedback on Lab Report 1)
GitHub repository: https://github.com/citgitfr/BenazzoVenkatadri_ThinShellSims/tree/main

I. Project Description and Goal

A. Problem Statement

Thin-shell carbon fiber reinforced polymer (CFRP) structures are widely used in deployable spacecraft structures, such as the collapsible booms that deploy solar arrays, antennas, and solar sails [1-3]. A large number of cross-sections are available for these booms, e.g. tape springs, triangular rollable and collapsible (TRAC) booms, and omega booms [4-5], and the booms are subjected to a correspondingly wide range of loads, including bending, twisting, buckling, coiling, and folding. Due to these complicated geometries and load cases, it is often difficult to choose the design (geometric parameters, CFRP material, and layup) that maximizes stiffness or buckling load, or minimizes mass or deflection, while satisfying constraints such as material strength. The goal of this project is to create a team of Claude agents that, for a given load case and objective, determines the best material, layup, and geometric parameters for a CFRP structure by running OpenSeesPy finite element models [6].

B. Related Work

Several studies have optimized the geometry of deployable composite booms. Previous authors have performed a multi-objective optimization of the cross-sectional parameters of a lenticular boom with a genetic algorithm, minimizing the coiling radius, coiling moment, and mass per unit length while maximizing the moments of area, subject to material strength constraints, and selected a design from the Pareto front that improved on their initial sample in every objective [7-8]. Other work has trained neural-network surrogates on finite element results of a single boom geometry, varying its radii and angles to maximize buckling strength, stiffness, or resistance to failure during coiling [9], and performance-driven design methods have been developed for collapsible tube masts [10]. These methods speed up the optimization of a few geometric parameters, but they do not optimize the material and layup of the laminate, nor do they compare different cross-section shapes. This gap is likely due to the effort required to generate a new geometry and mesh for each cross-section, and to convergence difficulties that arise when changes in material properties alter how buckles form [11].

OpenSeesPy [6] is the Python interface to the open-source OpenSees finite element framework. Although it is used mostly for earthquake engineering, its documentation [12] describes the features required for thin composite shells: the `ElasticOrthotropic` material for each ply, the `LayeredShell` section for the laminate, and the `ASDShellQ4` element, whose corotational formulation captures large displacements and rotations. No literature was found on the use of OpenSeesPy for thin-shell CFRP structures, so this project will address that gap in the literature.

Large language models (LLMs) have recently been applied to structural analysis. Liu *et al.* [13] found that the accuracy of LLMs solving beam problems directly decreases as the problems move away from canonical cases, and that reliability improves greatly when the task is reframed as writing and running OpenSeesPy code, with the results visualized for the user. Zhang *et al.* [14] developed a team of specialized agents that write, run, and validate OpenSeesPy programs for steel design, and Ni and Buehler [15] showed that multi-agent LLM teams can solve elasticity problems by writing and correcting finite element code. No study to date combines LLM agents, OpenSeesPy, and the optimization of thin-shell composites; this is another gap addressed by the project.

II. Why an Agent

The design process requires repeated decisions: after each analysis, a new combination of material, layup, and geometry must be chosen to improve the response to the given load case, and an agent can carry out this iteration. A single prompt to an LLM in a chat interface is insufficient, because the response of each design must be computed with a structural analysis tool such as OpenSeesPy rather than estimated by the LLM [13]. A conventional optimizer, such as MATLAB's `fmincon`, could handle the continuous geometric parameters, but the materials and practical layups are discrete, categorical choices that do not fit easily into a single objective function with constraints. Finally, a fixed script is insufficient, because each geometry (which determines the mesh), each set of material properties and layup, and each load case requires a different OpenSeesPy model (and associated Python script) to be written.

III. Agent Design

A. System Architecture

The proposed system is shown in Fig. 1. Three LLM agents (red) make the decisions that require judgment, while deterministic tools (blue) generate, run, and post-process the finite element models. The user specifies the class of cross-section, the boundary conditions and loads, the objective, and the constraints. The orchestrator selects an initial design from the libraries compiled by the librarian, calls the tools to build and run the OpenSeesPy model, reads the objective and constraint values, plots the results, and selects the next design to optimize the behavior of the structure. At regular intervals, the Abaqus verifier converts the current model into an Abaqus

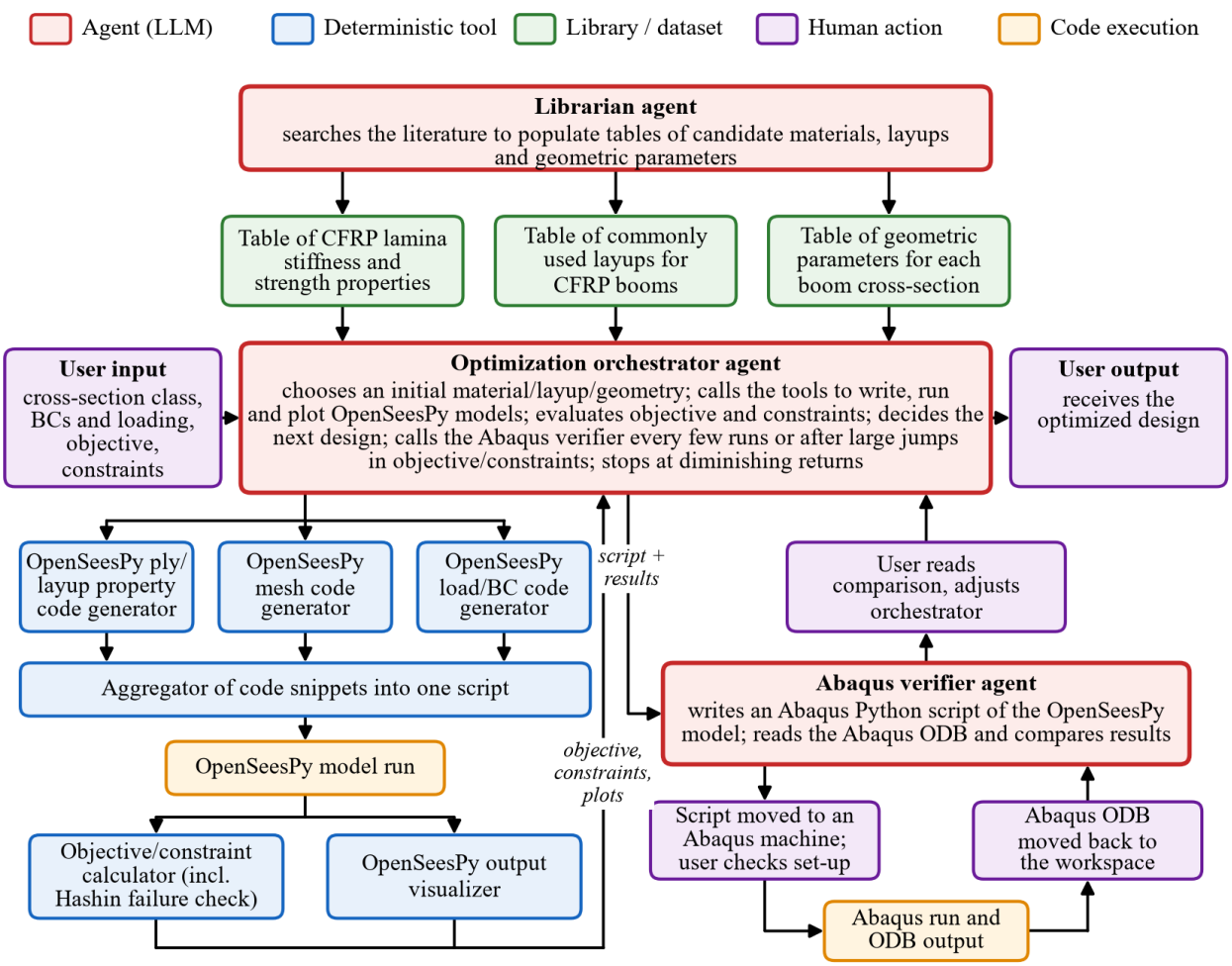


Fig. 1. Proposed agent system. Agents are Claude-driven; tools are deterministic Python scripts. The left branch builds and runs the OpenSeesPy model; the right branch is the Abaqus verification loop, in which the Abaqus run is performed by the user on a separate machine.

script that the user runs on a separate machine, since Abaqus is not installed in the class workspace; the verifier then compares the two sets of results.

B. Role of Each Agent and Tool

Librarian agent: compiles the catalog of candidate optimization variables (materials, layups, and geometric parameters) from the literature. Without it, a human would need to search the literature and build the catalog manually.

Optimization orchestrator agent: takes the user inputs and the libraries, calls the tools to write, run, and visualize OpenSeesPy models, calls the Abaqus verifier at key points, decides how to iterate, and returns the final design. It is the agent that coordinates the user, libraries, tools, and verifier; without it, the entire system would collapse.

Abaqus verifier agent: provides external verification by converting the OpenSeesPy model into an Abaqus model, which is easier for the users to inspect, and by comparing the OpenSeesPy and Abaqus results. Without it, there would be no way to confirm that the models created by the orchestrator represent the problem the user intended. It is separate from the orchestrator because, as discussed in class, the agent that verifies a model cannot be the agent that created it.

Code generators: each write an independent part of the OpenSeesPy script for ply/layup properties, mesh, and loads/boundary conditions. This specialization allows each to be tested individually: the laminate against classical laminate theory (CLT), the mesh with a convergence study, and the loads and boundary conditions by visual inspection.

Code aggregator: combines the three snippets into one cohesive script; without it, there is no runnable model.

Output visualizer: produces plots of the deformed structure and stress fields. Removing it would not break the pipeline, but it would make it much more difficult for the user to understand how the structure deforms.

Objective/constraint calculator: computes the objective and the constraint values, including the ply-by-ply Hashin failure criterion [16] (Tsai–Wu-type quadratic criteria [17] are an alternative). Without it, the LLM would need to compute these values itself, which LLMs do unreliably [13], so the optimization would likely be inaccurate.

C. Current Working Modules

Originally, separate agents were built for the librarian, the ply and layup properties (with CLT checks), the mesh, loads, and boundary conditions, the visualization, the material failure

check, and the conversion to Abaqus. After the discussion of agents versus tools in Lecture 8, the modules that execute a deterministic pipeline were converted to Claude Code *skills* (a Python script plus instructions), leaving only the orchestrator, librarian, and Abaqus verifier as agents. Each module was tested when it was created, and the test cases below were re-run after the reorganization.

Material library: The librarian has compiled 37 unidirectional lamina entries (carbon, glass, basalt, and flax fibers in epoxy and thermoplastic matrices), each with stiffness, strength, and ply thickness and a per-entry source note; 25 entries have the complete stiffness and strength data needed for both the laminate model and the failure check. The catalog is stored in `.claude/memory.material_property_catalog/`, and four entries are shown in Table I.

TABLE I
EXCERPT OF THE LAMINA PROPERTY LIBRARY

ID	Material [Ref.]	$E_1/E_2/G_{12}$ (GPa)	ν_{12}	$X_t/Y_t/S_{12}$ (MPa)
L001	IM7/8552 [18]	158.5/8.95/4.67	0.316	2501/64.0/91.1
L002	IM7/8552 [19]	171.4/9.08/5.29	0.32	2326/62.3/92.3
L013	AS4/3501-6 [20]	126/11/6.6	0.28	1950/48/79
L024	CFRP [7]	80.1/6.67/2.93	0.344	1063/66.0/126.8

Ply and layup property generator: The generator was tested on laminates for which ABD matrices had previously been computed by hand in the Caltech course Ae 165a. Table II compares the hand (CLT) values for an unsymmetric [90/0] laminate (E_1 = 140 GPa, E_2 = 10 GPa, G_{12} = 6 GPa, ν_{12} = 0.34, 1 mm plies) with the stiffness extracted from the OpenSeesPy model. The **A** and **B** matrices agree to machine precision; **D** is 0.39% low because `LayeredShell` integrates each sub-layer at its mid-plane, and the error falls to 0.02% with 32 sub-layers. For four symmetric four-ply layups of 54 µm plies, **D** agreed within 0.06–0.17% (`Code for Project/10-23-26 (Tara)/`).

TABLE II
ABD TERMS OF A [90/0] LAMINATE (N/MM, N, N·MM)

Term	CLT (hand)	OpenSeesPy	Difference
$A_{11} = A_{22}$	151 248.9	151 248.9	$< 10^{-12}$
A_{12}	6 856.6	6 856.6	$< 10^{-12}$
A_{66}	12 000.0	12 000.0	$< 10^{-12}$
$B_{11} = -B_{22}$	65 541.2	65 541.2	$< 10^{-12}$
$D_{11} = D_{22}$	50 416.3	50 219.4	−0.39%

Mesh, load, and boundary condition generator: A 1000 × 100 × 0.1 mm aluminum cantilever plate under a tip line load was used. For small loads, the tip deflection was within 1.2–2.8% of beam theory and the bending stress within 0.06%. For large loads (Fig. 2), the corotational `ASDShellQ4` model agreed with the analytical elastica solution within 0.2–2% up to a tip drop of 75% of the length, provided that the wide-plate stiffness $EI/(1-\nu^2)$ was used; 5 and 10 mm meshes agreed within 0.1% (`Code for Project/10-22-26 (Federico)/`).

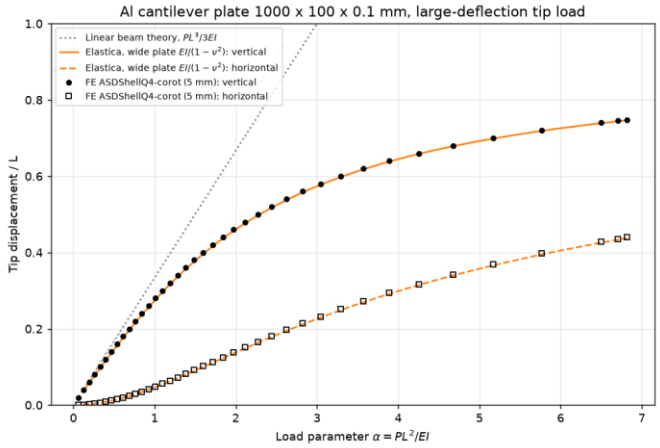


Fig. 2. Large-deflection test of the aluminum cantilever plate: tip displacements from OpenSeesPy (markers) and the elastica (line) versus the load parameter $\alpha = PL^2/EI$.

Integration of the laminate and mesh generators: A $[0_2/90_2]_s$ laminate (E_1 = 80 GPa, E_2 = 2 GPa, ν_{12} = 0.25, 1 mm plies) under a pure moment M_x = 50 N·m/m gave a curvature of 1.6670×10^{-2} m^{-1}, within 0.06% of the hand solution from Ae 165a. The aluminum cantilever was then rebuilt with this laminate (1000 × 100 × 8 mm, tip load P = 5 N; Fig. 3). The tip deflection of −5.563 mm is within 0.17% of laminated beam theory (0.03% after adding transverse shear), and the top-surface stress is within 0.06% of CLT (`Code for Project/10-26-26 (Federico and Tara)/plate_tip_load_laminate.py`).

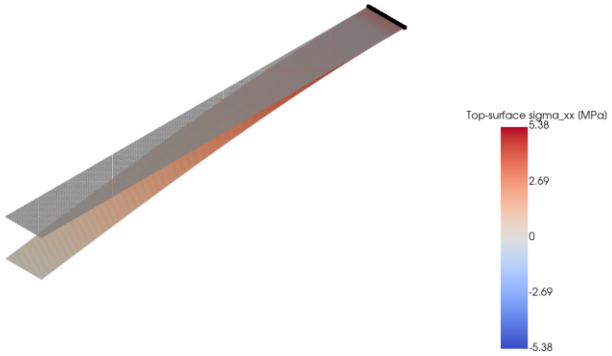


Fig. 3. $[0_2/90_2]_s$ CFRP cantilever plate under a tip load, colored by the top-surface stress σ_{xx} (deformation scaled ×18; gray: undeformed shape; black: clamped edge).

Conversion to Abaqus: The same cantilever was converted to an Abaqus/CAE Python script, which was run on a separate server. The model set-up was checked in the Abaqus interface, and the Abaqus tip deflection of −5.562 mm (Fig. 4) matches the OpenSeesPy value within 0.02% (`Code for Project/10-26-26 (Federico and Tara)/abaqus_tip_load_laminate.py`).

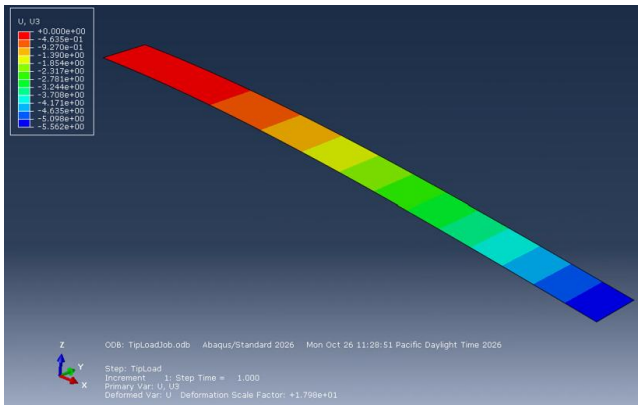


Fig. 4. Abaqus/Standard result for the CFRP cantilever of Fig. 3: vertical displacement U3 (mm), with a tip value of −5.562 mm.

Regression after reorganization: A regression suite (`.claude/skills/regression-tests/`) re-ran all of the above test cases with the new skills, together with new checks of the Hashin criterion against closed-form CLT ply stresses (within 0.4%). All results match the original modules, except for the final step of one large-deflection sweep, at which the Newton solver stops converging in both versions. The suite also revealed that off-axis plies on curved shells would have been misoriented by up to 15° in the original modules; this was corrected, and no earlier result was affected because all earlier models were flat.

IV. Evaluation

A. Verification Independent of the LLM

Each of the deterministic tools has been (or will be, for the modules not yet written) checked against analytical or human-generated finite element solutions, so their outputs can be trusted. The users will inspect the Abaqus translation of the finite element model at several points during the optimization and at its end, to confirm that the OpenSeesPy model represents the intended problem, and the verifier will check that the Abaqus and OpenSeesPy results are consistent. Success is defined as the design of a CFRP boom that satisfies the material failure criteria and the user-specified constraints, together with a record of an iteration process that progressively improves the objective until the improvements diminish.

B. Baselines

First, the results will be compared with the geometry-only optimizations of CFRP booms in the literature [7-9], with the aim of matching or improving on their designs. Second, a problem with a known optimum will be posed, such as maximizing the bending stiffness of a tape spring of fixed mass, for which the widest and thinnest laminate in the library is the solution. Third, the agent must be faster than a manual grid search over materials, layups, and geometries: writing each finite element model by hand takes a few hours, and analyzing it and choosing the next design takes another one to two hours. Similarly, the librarian should be faster than a human searching the literature for material properties.

V. Plan and Risks

A. Milestones

By Lab Report 3 (November 15), all tools, libraries, and agents will be built and tested individually, and the full pipeline will be tested end to end. By Lab Report 4 (November 29), the optimizer will be tested on a few simple cases to compare its performance with the baselines, and then on more complicated cases to demonstrate its capabilities.

B. Risks and Questions for the TAs

The main question is how to establish a baseline: an optimization over materials, layups, and geometries of this breadth has not been reported in the literature, so it is not clear how to evaluate whether the agent outperforms a baseline. A risk is that the optimization may not converge, or may converge to a local rather than a global optimum; if this occurs, a global method, such as a genetic algorithm seeded with several combinations from the libraries, may be incorporated. Also, even after the behavior of the agents has been verified on simple cases, hallucination is difficult to prevent in larger cases, especially within an optimization loop in which not every iteration is checked.

VI. Summary of Progress

As described in Section III-C, the ply/layup, mesh, load/boundary condition, visualization, and failure-check modules have been developed and verified against analytical solutions, hand calculations, and Abaqus, and the librarian and preliminary Abaqus verifier agent have been created. The objective/constraint calculator, the layup and geometry libraries, and the optimization loop of the orchestrator remain to be completed, and the Abaqus verifier agent must be modified to include the comparison of the OpenSeesPy output and Abaqus output database.

VII. Running Log and Repository

The running log and the GitHub repository are maintained continuously. Dated log entries for each team member are in the `Running Log` folder of the repository, the code for each test case is in `Code for Project`, the agents are in `.claude/agents`, and the Claude conversations used to build the modules are in `Claude Conversations`.

AI Use Acknowledgment

Claude Code (Opus 5.5, Anthropic) was used to create all Python scripts and agents and to run the test cases. It was also used to write the draft of this report from detailed outlines provided by the authors (and to create a typed version of the previously-handwritten flowchart), and to identify and check the bibliographic details of additional references. The authors read all agent .md files to ensure that they matched the desired functionality, checked the test cases against analytical calculations and Abaqus models, and proofread and edited the final report to ensure that it conveyed the intended information.

REFERENCES

[1] L. Puig, A. Barton, and N. Rando, "A review on large deployable structures for astrophysics missions," *Acta Astronaut.*, vol. 67, no. 1–2, pp. 12–26, 2010.

[2] T. W. Murphey, W. Francis, B. Davis, and J. M. Mejia-Ariza, "High strain composites," in *Proc. 2nd AIAA Spacecraft Struct. Conf.*, 2015, AIAA 2015-0942.

[3] J. M. Fernandez, "Advanced deployable shell-based composite booms for small satellite structural applications including solar sails," presented at the *Int. Symp. Solar Sailing*, Kyoto, Japan, Jan. 2017.

[4] J. A. Banik and T. W. Murphey, "Performance validation of the triangular rollable and collapsible mast," in *Proc. 24th Annu. AIAA/USU Conf. Small Satellites*, Logan, UT, 2010, Paper SSC10-II-1.

[5] T. W. Murphey, D. Hunt, D. Browning, and M. W. Folkers, "Performance comparison of flattenable and rollable booms," in *AIAA SciTech 2026 Forum*, 2026, AIAA 2026-0802.

[6] M. Zhu, F. McKenna, and M. H. Scott, "OpenSeesPy: Python library for the OpenSees finite element framework," *SoftwareX*, vol. 7, pp. 6–11, 2018.

[7] T.-W. Liu, J.-B. Bai, and N. Fantuzzi, "Folding behavior of the thin-walled lenticular deployable composite boom: Analytical analysis and many-objective optimization," *Mech. Adv. Mater. Struct.*, vol. 30, no. 11, pp. 2221–2239, 2023.

[8] J. Sun, Q. Han, X. Cheng, H. Shi, R. Ding, M. Shi, and C. Liu, "A data-driven computational optimization framework for designing thin-walled lenticular deployable composite boom with optimal load-bearing and folding capabilities," *Thin-Walled Struct.*, vol. 203, p. 112244, 2024.

[9] F. Palmeri and S. Laurenzi, "Neural network-based surrogate modeling for buckling performance optimization of lightweight-composite collapsible tubular masts," *Biomimetics*, vol. 9, no. 8, p. 494, 2024.

[10] F. Palmeri and S. Pellegrino, "Performance-driven design of deployment mechanism for collapsible tube masts," in *AIAA SciTech 2026 Forum*, 2026, AIAA 2026-1041.

[11] W. Luo and S. Pellegrino, "Formation and propagation of buckles in coilable cylindrical thin shells with a thickness discontinuity," *Int. J. Solids Struct.*, vol. 259, p. 112010, 2022.

[12] "The OpenSeesPy Library," OpenSeesPy documentation. [Online]. Available: https://openseespydoc.readthedocs.io/en/latest/

[13] J. Liu, Z. Geng, R. Cao, L. Cheng, P. Bocchini, and M. Cheng, "A large language model-empowered agent for reliable and robust structural analysis," *Struct. Infrastruct. Eng.*, pp. 1–16, 2026.

[14] Z. Zhang, K. Chen, C. B. Cevik, and C. D. Moen, "LLM agent framework for structural steel design," in *Proc. Annu. Stability Conf.*, Structural Stability Research Council, 2026.

[15] B. Ni and M. J. Buehler, "MechAgents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge," *Extreme Mech. Lett.*, vol. 67, p. 102131, 2024.

[16] Z. Hashin, "Failure criteria for unidirectional fiber composites," *J. Appl. Mech.*, vol. 47, no. 2, pp. 329–334, 1980.

[17] S. W. Tsai and E. M. Wu, "A general theory of strength for anisotropic materials," *J. Compos. Mater.*, vol. 5, no. 1, pp. 58–80, 1971.

[18] National Center for Advanced Materials Performance, "Hexcel 8552 IM7 unidirectional prepreg qualification material property data report," Wichita State Univ., Wichita, KS, Rep. CAM-RP-2009-015 Rev. A, 2011.

[19] P. P. Camanho, P. Maimí, and C. G. Dávila, "Prediction of size effects in notched laminates using continuum damage mechanics," *Compos. Sci. Technol.*, vol. 67, no. 13, pp. 2715–2727, 2007.

[20] P. D. Soden, M. J. Hinton, and A. S. Kaddour, "Lamina properties, lay-up configurations and loading conditions for a range of fibre-reinforced composite laminates," *Compos. Sci. Technol.*, vol. 58, no. 7, pp. 1011–1022, 1998.